MANUAL TÉCNICO DEL SISTEMA

LAVA RÁPIDO VEHICULAR

Arquitectura, componentes, datos, API, seguridad y mantenimiento

José Stiven Perdomo Martínez

Santiago Gordo Pérez

Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software

Ficha 3145555

Colombia · 2026

Resumen

El presente manual técnico describe la estructura y los mecanismos de funcionamiento de Lava Rápido Vehicular, plataforma integrada por una aplicación móvil para clientes y operadores, un panel administrativo web, un backend Java/Spring Boot y una base de datos PostgreSQL versionada con Liquibase. Se explican su arquitectura monolítica modular, responsabilidades de componentes, contratos principales de API, reglas de reservas, verificación de pagos Wompi, seguridad JWT, notificaciones, despliegue y mantenimiento.

El contenido refleja las características identificadas en el estado técnico de los repositorios con referencia al 4 de noviembre de 2026 y conserva los ocho diagramas técnicos del manual previo. Las tecnologías y estructuras declaradas no equivalen por sí solas a una comprobación de todas las integraciones en producción. No se incluyen contraseñas, tokens, claves ni datos personales.

Palabras clave: arquitectura de software, Spring Boot, API REST, PostgreSQL, Liquibase, seguridad JWT, Wompi.

Control documental

Campo	Descripción
Documento	Manual técnico de Lava Rápido Vehicular
Versión documental	Edición revisada · 4 de noviembre de 2026
Finalidad	Consulta técnica, mantenimiento y entrega formativa
Alcance	Aplicación móvil, panel web, API backend, esquema de datos e integraciones
Limitación	El esquema declarado y las configuraciones no demuestran automáticamente la versión o disponibilidad del despliegue actual

Contenido

1. Introducción, alcance y arquitectura

2. Tecnologías y organización interna

3. Módulos funcionales y contratos API

4. Modelo de datos y persistencia

5. Gestión técnica de reservas

6. Integración de pagos mediante Wompi

7. Seguridad, autenticación y autorización

8. Construcción y despliegue

9. Notificaciones y procesos asíncronos

10. Calidad, mantenimiento y recuperación

11. Limitaciones y conclusiones

Referencias y anexos

1. Introducción, alcance y arquitectura

Lava Rápido Vehicular organiza el catálogo de servicios, vehículos, reservas, cobros y trabajo de operadores del autolavado. La aplicación móvil permite gestionar cuentas y reservas, pagar y consultar notificaciones; también ofrece un recorrido para personal operador. El panel web permite administración y supervisión. Los dos clientes interactúan con la API y no se conectan directamente a PostgreSQL.

1.1 Objetivo

Brindar una referencia técnica mantenible para comprender la arquitectura, los componentes, el modelo de persistencia, las interfaces de programación, las reglas de negocio y las operaciones de soporte del sistema.

1.2 Alcance y exclusiones

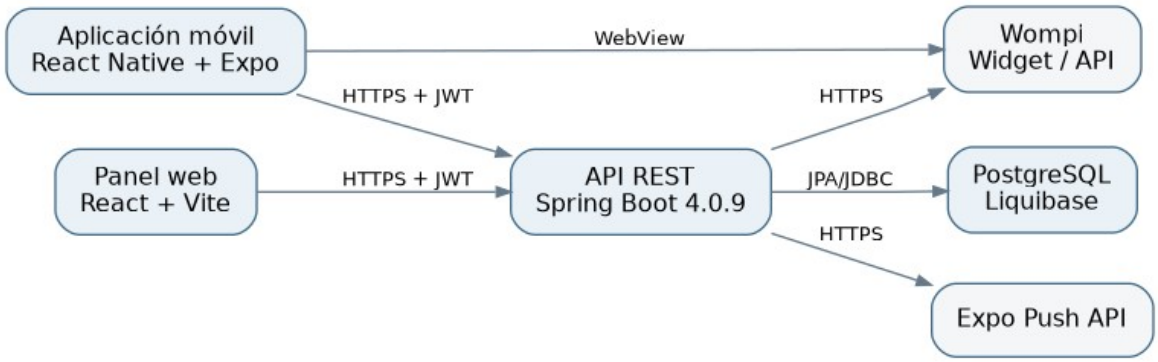
Incluye autenticación, usuarios, catálogo, marcas, vehículos, reservas, operadores, asignaciones, Wompi, calificaciones, notificaciones, auditoría y controles de seguridad. Quedan fuera del alcance el rastreo GPS de operadores, la optimización de rutas, el inventario físico y la contabilidad detallada. Las vistas de mapa presentan puntos de atención y no seguimiento de personal.

1.3 Estilo arquitectónico

La solución responde a una arquitectura cliente-servidor con un backend monolítico modular. Los módulos de dominio están organizados por paquetes dentro de un único servicio Spring Boot; no constituyen microservicios desplegables por separado. El backend utiliza controladores REST, servicios de negocio, repositorios Spring Data JPA, entidades y DTO. Spring Security aplica controles transversales de autenticación y autorización.

Figura 1

Arquitectura general de Lava Rápido Vehicular



Nota. Diagrama conservado y adaptado del manual técnico proporcionado. Ilustra conexiones lógicas, no certifica disponibilidad de proveedores.

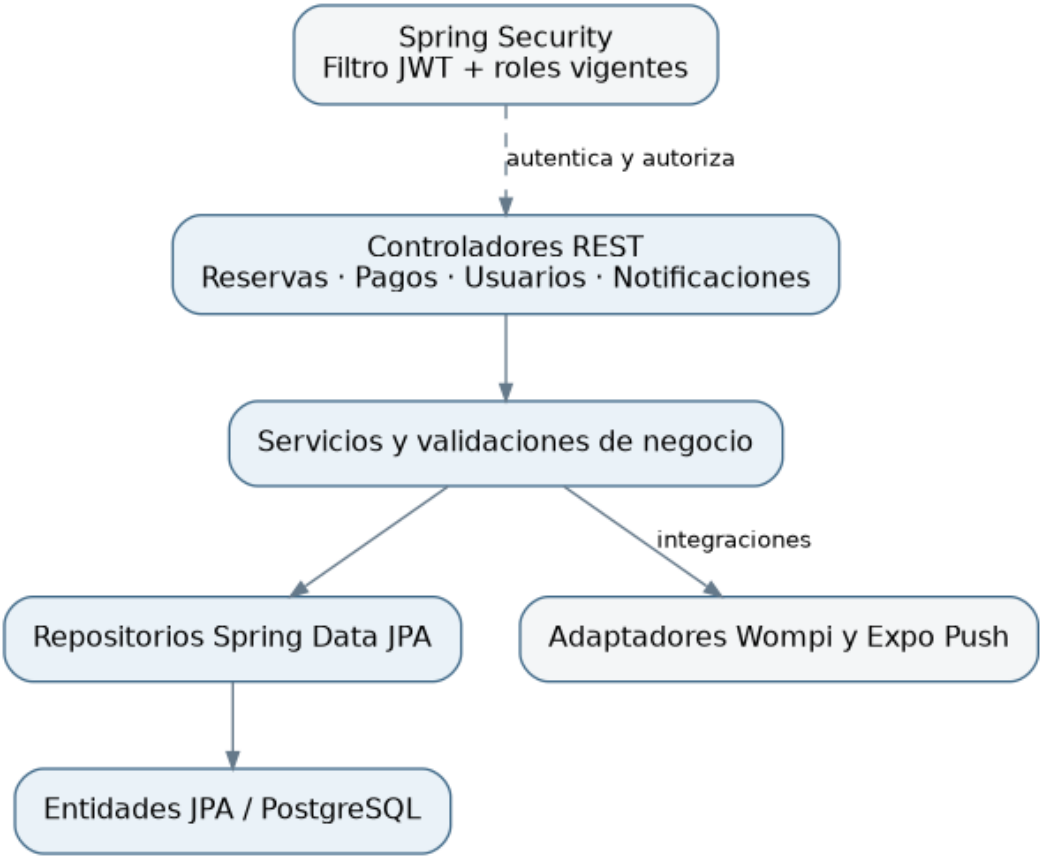
2. Tecnologías y organización interna

Capa	Tecnología declarada	Función
Aplicación móvil	Expo ~58.0.20; React Native 0.88.0; React 19.3.0; TypeScript	Interfaz de cliente y operador; navegación y consumo de API
Panel administrativo	React 19; Vite 6; React Router 7; Zustand 5	Gestión administrativa y estado de sesión
Backend	Java 17; Spring Boot 4.0.9; JJWT 0.12.6; Springdoc 3.0.2	Reglas de negocio, API REST, JWT e integraciones
Persistencia	PostgreSQL 16; Spring Data JPA; Liquibase 5.0.2 declarado	Datos transaccionales e historial de migraciones
Externos	Wompi; SMTP; Expo Push API	Cobros, recuperación y avisos opcionales
Empaquetado	Docker/Compose y configuración Expo/EAS	Ejecución backend y preparación de compilaciones móviles

Las versiones se basan en manifiestos declarados de la aplicación y en el entorno de base descrito. Una biblioteca puede evolucionar en una versión posterior; cualquier entrega debe vincularse a una revisión Git concreta y comprobar sus dependencias.

Figura 2

Organización del backend por responsabilidades



Nota. Se mantienen controladores, servicios, repositorios, entidades y filtros de seguridad del diagrama de referencia.

2.1 Patrones técnicos utilizados

Patrón o práctica	**Ejemplo de uso**	**Ventaja**
Controller–Service–Repository	ReservaController → ReservaService → ReservaRepository	Separa exposición HTTP, reglas de negocio y acceso a datos
Repository	Interfaces Spring Data para reservas, pagos y usuarios	Aísla consultas y persistencia
DTO	ReservaRequestDTO y ReservaResponseDTO	Delimita entradas y respuestas HTTP
Inyección de dependencias	Servicios Spring y cliente Wompi por interfaz	Permite sustituir colaboradores y facilita pruebas
Cliente de integración	WompiTransactionClient / WompiTransactionClientImpl	Encapsula solicitudes al proveedor
Componentes / Hooks	Pantallas React y useWompiWidget	Reutiliza componentes y lógica de UI
Rutas protegidas	RequireAdmin; seguridad API	Restringe navegación y, en el servidor, operaciones

3. Módulos funcionales y contratos API

Los endpoints siguientes identifican familias y acciones principales. Sus parámetros, cuerpos DTO y códigos HTTP completos deben contrastarse con los controladores de la revisión que se vaya a desplegar. Los accesos dependen del JWT y del rol o propietario del recurso, salvo las rutas públicas expresamente permitidas.

Dominio	Componentes relevantes	Responsabilidad	Persistencia
Usuarios / autenticación	UserController; PasswordResetController	Registro, login, perfil y recuperación	users; roles; user_roles; tokens_recuperacion
Servicios	ServicioController; ServicioService	Catálogo, búsqueda y activación	servicios
Marcas y vehículos	MarcaController; VehiculoController	Marcas y vehículos asociados al usuario	marcas; vehiculos
Reservas	ReservaController; ReservaService	Agenda, estado, cancelación y reglas de horario	reservas
Operadores / asignaciones	OperadorController; AsignacionController	Personal, tareas y avances del servicio	operadores; asignaciones
Pagos	PagoController; PagoService	Intentos, widget, verificación, webhook y conciliación	pagos; pago_intentos
Calificaciones	CalificacionController	Puntuación de reserva finalizada	calificaciones
Notificaciones	NotificacionController; NotificacionService	Historial, lectura y dispositivos	notificaciones; dispositivos_push; notificacion_entregas
Trazabilidad	AuditoriaController; LogErrorController	Consultas y registros específicos de auditoría y errores	auditoria; log_errores

3.1 API REST: operaciones representativas

Método	Ruta ilustrativa	Acceso	Función
POST	/api/users/register	Pública	Registrar cliente
POST	/api/users/login	Pública	Iniciar sesión
POST	/api/auth/forgot-password	Pública con controles	Solicitar restablecimiento
GET	/api/servicios	Según recurso	Consultar catálogo
POST	/api/reservas	Usuario autorizado	Crear reserva
PATCH	/api/reservas/{id}/cancelar	Propietario y reglas	Cancelar reserva elegible
POST	/api/pagos/reserva/{id}	Propietario o ADMIN	Iniciar intento de cobro
POST	/api/pagos/reserva/{id}/verificar	Actor autorizado	Verificar transacción con backend
POST	/api/asignaciones	ADMIN	Asignar operador a reserva
GET	/api/asignaciones/mis-asignaciones	Operador	Consultar tareas propias
PATCH	/api/asignaciones/{id}/estado	Operador autorizado	Actualizar ejecución
POST	/api/calificaciones	Propietario elegible	Calificar servicio finalizado
GET	/api/notificaciones	Usuario autenticado	Consultar avisos

Los nombres de los roles del backend incluyen ADMIN, USER y OPERATOR. La API es la autoridad final de permisos: ocultar botones o páginas en React no reemplaza la autorización por rol y por propietario del recurso.

3.2 Clientes y administración

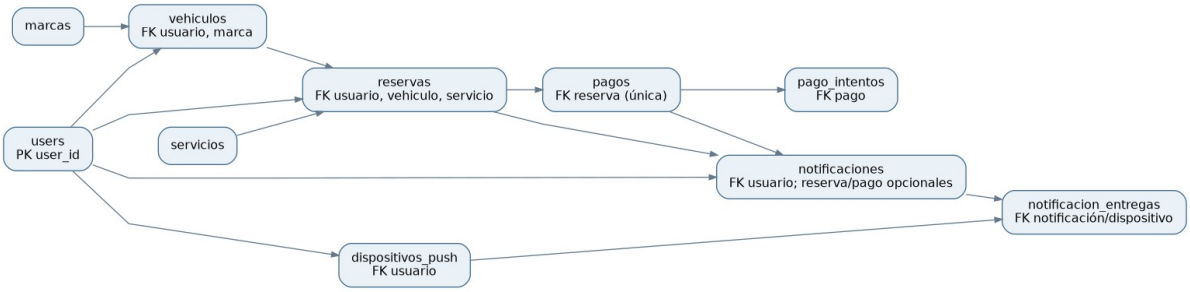
El móvil agrupa navegación de cliente y operador con AppNavigator y OperatorNavigator. El panel web utiliza React Router y RequireAdmin; las interfaces incluyen panel principal, servicios, marcas, vehículos, ubicaciones, operadores y turnos. Las acciones de cobro administrativo se integran con reservas; no debe afirmarse que existe un módulo CRUD de pagos independiente si no aparece en las rutas.

4. Modelo de datos y persistencia

PostgreSQL recibe operaciones únicamente desde el backend mediante JPA/Hibernate. Los cambios estructurales se declaran en migraciones Liquibase; la configuración ddl-auto=validate indica validación del esquema, no creación automática como mecanismo principal. Se identifican 18 tablas de aplicación y 24 changesets activos declarados en los archivos; su ejecución en una instancia desplegada exige revisar DATABASECHANGELOG.

Figura 3

Modelo resumido de relaciones de datos



Nota. Vista resumida conservada del manual previo. La lista de 18 tablas se documenta a continuación.

4.1 Tablas por dominio

Dominio	Tablas declaradas	Relación principal
Identidad	roles, users, user_roles, tokens_recuperacion	Roles N:M con usuarios; recuperación asociada a cuenta
Catálogo	servicios, marcas, vehiculos	Vehículo pertenece a usuario y marca
Operación	reservas, operadores, asignaciones	Reserva de vehículo y servicio; asignación de operador
Pagos	pagos, pago_intentos	Pago único por reserva; múltiples intentos posibles
Calidad / control	calificaciones, auditoria, log_errores	Evaluaciones y eventos técnicos
Avisos	notificaciones, dispositivos_push, notificacion_entregas	Avisos por usuario y trazabilidad de entrega

4.2 Claves y restricciones relevantes

- Las claves primarias utilizan principalmente UUID y las relaciones principales se protegen mediante claves foráneas.
- La tabla reservas almacena precio_pactado y duracion_minutos_pactada, preservando los valores acordados.

- Una restricción PostgreSQL de exclusión GiST, declarada con btree_gist, previene el solapamiento de reservas no canceladas del mismo vehículo.
- Las tablas pagos, asignaciones y calificaciones limitan mediante unicidad la vinculación de un registro por reserva, según el caso.
- Los intentos de pago tienen referencia única, reglas parciales de unicidad por transacción/ambiente y restricción para intentos pendientes compatibles.
- Las notificaciones y entregas utilizan índices y claves para evitar duplicaciones de eventos y envíos por canal/dispositivo.

4.3 Procedimiento seguro de evolución

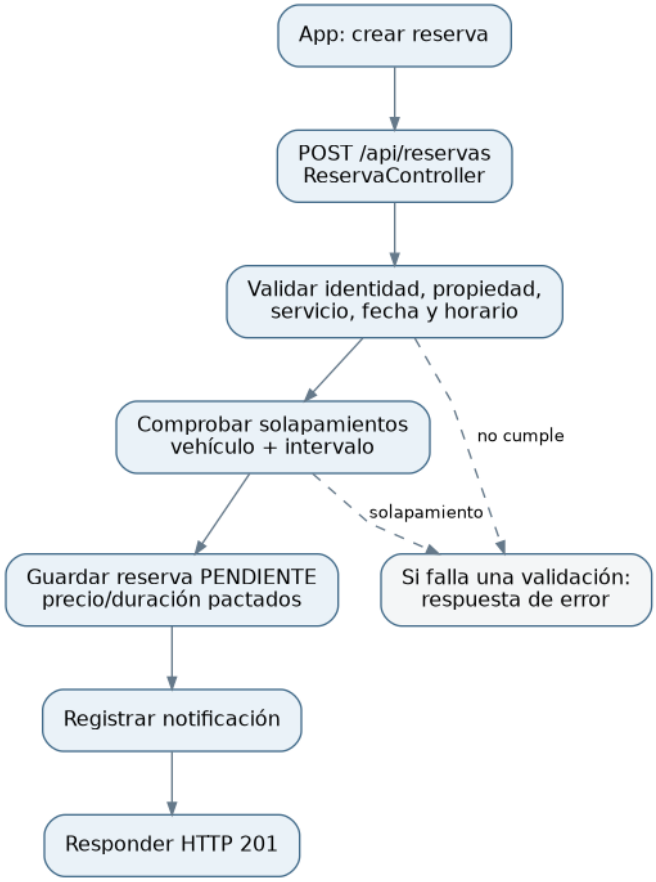
- Identificar changelog y esquema aplicado para el ambiente de destino.
- Registrar respaldo, plan de restauración y revisión de índices/restricciones.
- Ejecutar validate y status antes de aprobar update.
- Aplicar cambios únicamente con autorización, y verificar DATABASECHANGELOG, checksums y compatibilidad con JPA.
- Documentar versión/commit, resultados y acciones ante fallo.

5. Gestión técnica de reservas

El cliente envía vehículo, servicio y horario. ReservaController delega a ReservaService la comprobación de identidad, propiedad del vehículo, servicio activo, fecha, horario entre 06:00 y 20:00 y solapamientos. El backend guarda una reserva PENDIENTE con precio y duración pactados y puede crear una notificación; el resultado de éxito se devuelve con HTTP 201.

Figura 4

Flujo principal de creación de reserva



Nota. Diagrama del manual original conservado; las ramas de validación representan rechazos controlados.

El estado operativo de la reserva se gestiona de modo independiente del pago. Estados descritos: PENDIENTE, ASIGNADA, EN_PROCESO, FINALIZADA y CANCELADA. Las transiciones autorizadas dependen de las reglas de ReservaService y AsignacionService; una reserva pagada no debe interpretarse automáticamente como finalizada.

5.1 Ejecución con operador

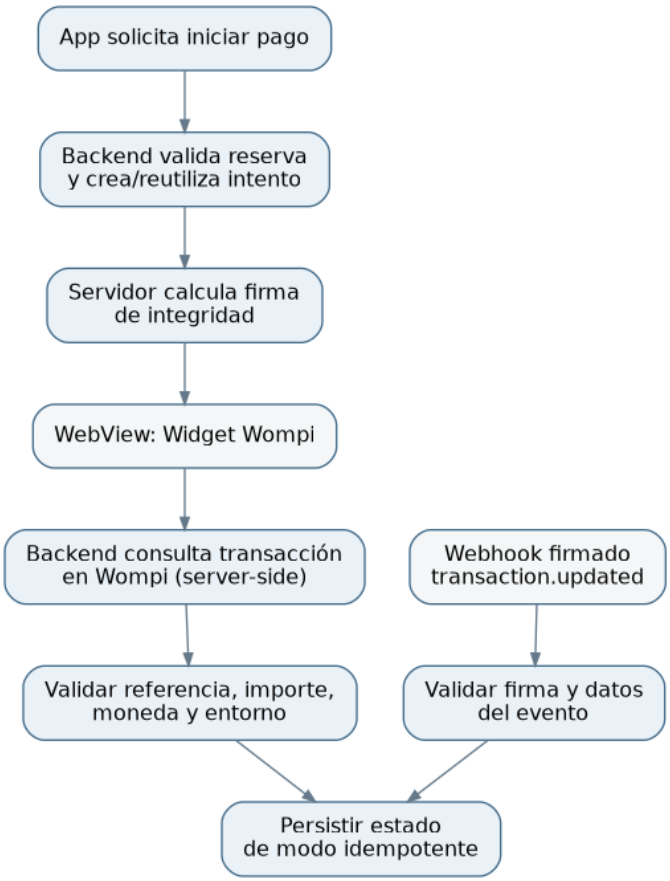
Un administrador asigna un operador activo a una reserva apta. El operador consulta únicamente sus asignaciones y cambia la ejecución conforme a las transiciones permitidas. La persistencia debe mantener coherencia entre asignación y estado de reserva; ante rechazos o errores de red, la interfaz debe volver a consultar el estado confirmado por el servidor.

6. Integración de pagos electrónicos Wompi

Para iniciar un pago, el backend comprueba la reserva, calcula el importe basado en el precio pactado y crea o reutiliza un intento. Entrega al cliente datos públicos para abrir el widget de Wompi; las credenciales privadas se conservan únicamente en el servidor. Un callback o redirección no equivale a aprobación: se requiere verificación desde la API o recepción de un evento firmado válido.

Figura 5

Inicio y confirmación de pagos mediante Wompi



Nota. El flujo remoto de verificación y el webhook son vías diferenciadas; el orden de llegada no es obligatorio.

6.1 Validaciones y estados de pago

- Verificar referencia, identificador remoto, ambiente, monto y moneda COP antes de aplicar resultados.
- Aceptar eventos transaction.updated únicamente después de validar integridad y coherencia con el intento local.

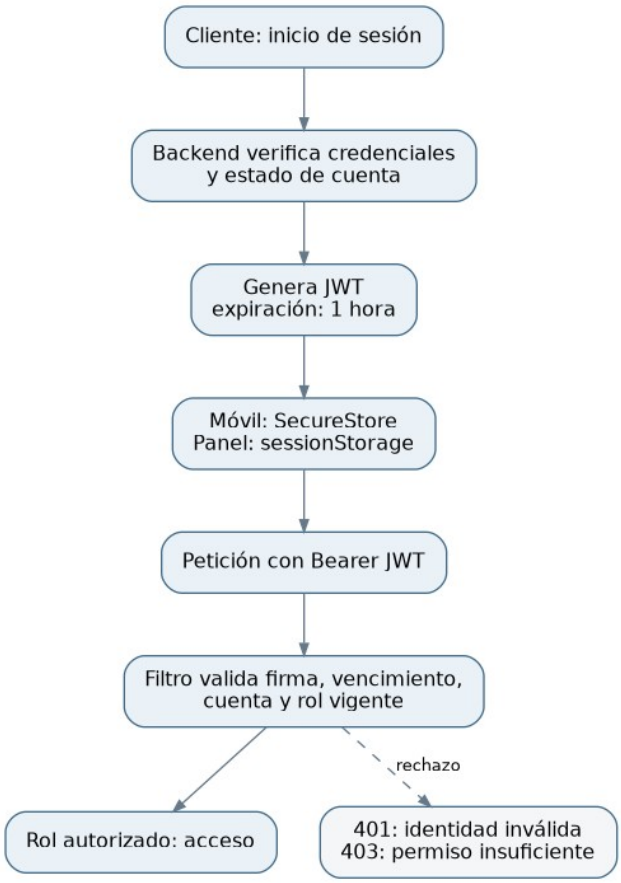
- Estados internos de pago: pendiente, aprobado y rechazado; el intento contempla también aprobado_duplicado para revisión.
- Aplicar lógica idempotente ante eventos repetidos y conservar historial de intentos.
- La reconciliación administrativa consulta al proveedor según las condiciones habilitadas y nunca confía únicamente en la presentación del cliente.

7. Seguridad, autenticación y autorización

Spring Security configura acceso stateless, token JWT bearer, restricciones por roles y validación del estado de la cuenta. La autenticación se realiza en el servidor; el frontend móvil utiliza SecureStore y el panel web sessionStorage para conservar información de sesión según su implementación. Esto obliga a prestar atención a XSS y al ciclo de vida del token en el navegador.

Figura 6

Flujo de autenticación y autorización JWT



Nota. Representación funcional conservada del manual de referencia. El servidor debe validar firma, expiración y autorización.

7.1 Controles complementarios

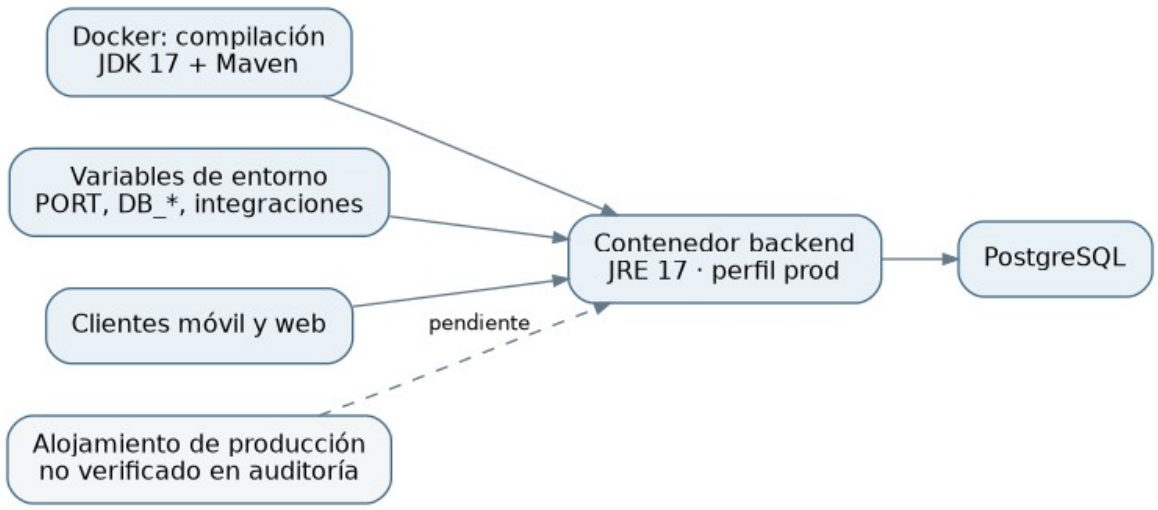
- Contraseñas almacenadas mediante BCrypt; restablecimiento con token temporal y de uso único.
- Limitación de solicitudes en rutas de autenticación (filtro de tasa): su contador en memoria por instancia no equivale a protección distribuida.
- HTTPS y CORS con orígenes explícitos para clientes públicos; no registrar valores de secretos en logs.
- Verificación del propietario en reservas, vehículos, pagos y calificaciones; restricciones a tareas propias para OPERATOR.
- No divulgar identificadores personales o información de transacciones en capturas, logs o documentación.

8. Construcción, configuración y despliegue

El repositorio backend utiliza Dockerfile con compilación Maven y ejecución Java 17. Se dispone de configuración Docker/Compose, variantes application*.properties y variables por nombre para DB, JWT, Wompi y correo. En móvil se declaran app.json, app.config.js y eas.json; el panel administrativo dispone de compilación Vite.

Figura 7

Empaquetado e integración de infraestructura



Nota. El diagrama conserva una vista conceptual; no prueba por sí mismo qué revisión está publicada.

8.1 Direcciones públicas documentadas

Componente	Dirección comunicada	Estado de verificación
Backend (Render)	https://lava-rapido-backend.onrender.com	Dirección documentada; no se realizó comprobación de disponibilidad en esta edición
Panel administrativo (Render)	https://lava-rapido-admin.onrender.com	Dirección documentada; no se verificó la versión desplegada
Aplicación Android	Distribución Expo/EAS mediante artefacto de compilación	La configuración de build no equivale a confirmar una publicación vigente

8.2 Variables de ambiente y dependencias

Entre las categorías de configuración se encuentran conexión a PostgreSQL (DB_*), autenticación JWT, orígenes CORS autorizados, ambiente Wompi, SMTP, habilitación de notificaciones, URL de API móvil

y URL de backend del panel. La documentación solo debe registrar nombres y finalidad de cada variable, nunca sus valores sensibles.

8.3 Verificaciones después del despliegue

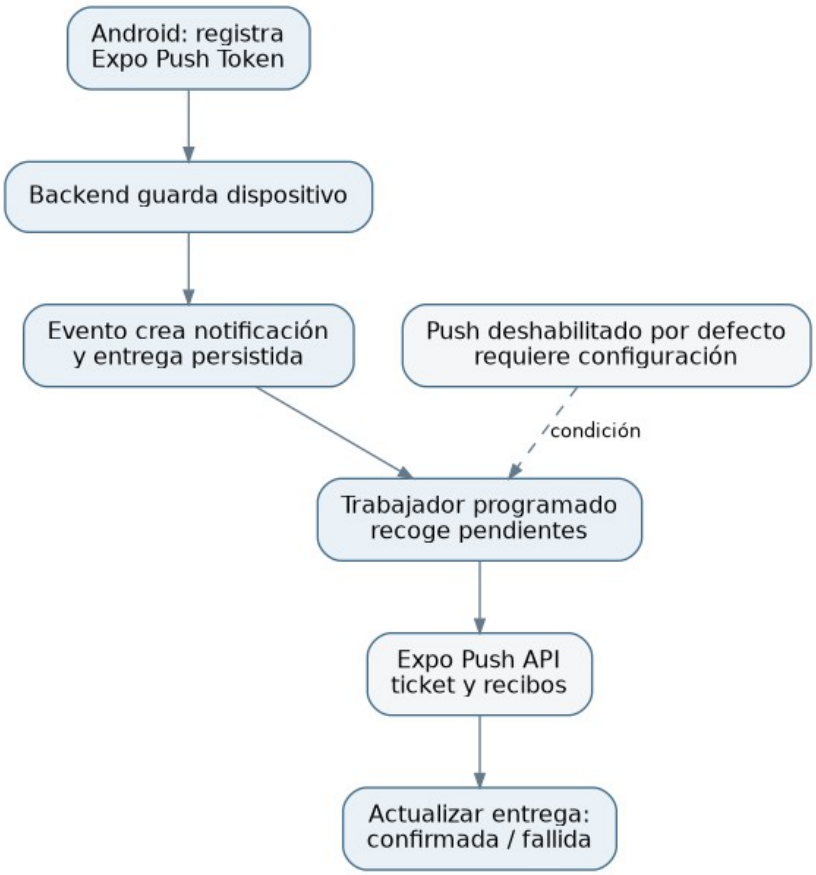
- Registrar repositorio, rama, commit y fecha de cada componente desplegado.
- Revisar certificados HTTPS y conectividad desde móvil y panel sin publicar tokens.
- Consultar endpoints de salud; interpretar readiness independientemente de un health general degradado.
- Confirmar conexión de backend con PostgreSQL y versión de esquema realmente aplicada.
- Ejecutar en ambiente autorizado los flujos de registro, reserva, pago sandbox, asignación y notificaciones con datos sintéticos.
- Conservar capturas anonimizadas y registros de respuesta depurados.

9. Notificaciones y procesamiento asíncrono

La aplicación móvil dispone de avisos internos: historial, contador de no leídas y acciones de marcado. En Android, cuando está habilitada la integración nativa, se puede registrar un dispositivo para Expo Push. El backend conserva notificaciones y registros de entrega; un proceso programado maneja pendientes, reintentos y respuestas del proveedor. La entrega push real depende de configuración y permisos del entorno.

Figura 8

Procesamiento de notificaciones push



Nota. La figura resume un mecanismo opcional, condicionado a habilitación y conectividad de Expo Push.

La bandera APP_NOTIFICACIONES_PUSH_ENABLED aparece deshabilitada por defecto en la configuración revisada. No debe confundirse la presencia de tablas y clases de entrega con evidencia de mensajes push recibidos en producción. El servicio de correo SMTP también exige una verificación independiente de envío y recepción.

10. Calidad, mantenimiento y recuperación

10.1 Pruebas disponibles y alcance

Existen suites de prueba en móvil, panel y backend. Su presencia en los repositorios no equivale a ejecución de la versión publicada. El informe de rendimiento local con Apache k6 del 23 de octubre de 2026 documentó 50 usuarios virtuales durante cinco minutos, 308,44 solicitudes de negocio por segundo, p95 de 201,74 ms y 0 % de errores en solicitudes autenticadas de solo lectura. Los resultados no deben extrapolarse al servidor Render ni a pagos o escrituras.

10.2 Mantenimiento recomendado

- Controlar dependencias, vulnerabilidades, actualizaciones y compatibilidad de clientes Android.
- Revisar migraciones Liquibase, integridad, índices y capacidad de restauración de PostgreSQL.
- Comprobar periódicamente el estado de integraciones Wompi y SMTP usando ambientes autorizados.
- Mantener un inventario de endpoints y permisos por versión; verificar transiciones y procesos idempotentes.
- Incorporar métricas JVM, GC, pool HikariCP y latencia para diagnóstico, sin publicar información sensible.
- Ejecutar pruebas de regresión antes de desplegar cambios en reservas, pagos, autenticación o asignaciones.

10.3 Respuesta ante incidentes

Ante una incidencia, identificar versión y ambiente, conservar logs depurados, clasificar gravedad, aislar integraciones cuando proceda y evaluar rollback o restauración con respaldo válido. No alterar migraciones ya aplicadas para solucionar inconsistencias; debe planearse una migración correctiva o recuperación aprobada.

11. Limitaciones y conclusiones

Este manual refleja tecnologías, estructuras y comportamientos presentes en la documentación técnica disponible. No se consultaron directamente instancias de PostgreSQL desplegadas, transacciones reales de Wompi, registros productivos actuales ni entrega real de mensajes push/correo durante la preparación de esta edición. Por ello, las rutas públicas, la arquitectura lógica y los cambiosets declarados no constituyen una certificación de disponibilidad del entorno.

Lava Rápido Vehicular dispone de una estructura cliente-servidor modular con mecanismos relevantes de seguridad, consistencia de reservas, verificación de pagos y persistencia controlada. La separación de responsabilidades y el uso de migraciones facilitan el mantenimiento. Para cerrar formalmente una versión técnica se requiere asociarla a commits, pruebas ejecutadas y evidencias de despliegue verificadas.

Referencias

Equipo Lava Rápido Vehicular. (2026a). Especificación de requisitos de software: Lava Rápido Vehicular [Documento interno].

Equipo Lava Rápido Vehicular. (2026b). Diseño del software: Interfaces, arquitectura, patrones y modelos de datos [Documento interno].

Equipo Lava Rápido Vehicular. (2026c). Procedimiento de implementación del software [Documento interno].

Equipo Lava Rápido Vehicular. (2026d). Informe técnico de pruebas de rendimiento con Apache k6 [Informe interno].

Equipo Lava Rápido Vehicular. (2026e). Manual técnico del sistema, edición 2.0 [Documento base proporcionado].

Oracle. (s. f.). Java documentation. https://docs.oracle.com/en/java/

Spring. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/

PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/

Liquibase. (s. f.). Liquibase documentation. https://docs.liquibase.com/

Wompi. (s. f.). Documentación para desarrolladores. https://docs.wompi.co/

Anexo A. Principales rutas de código de referencia

Responsabilidad	Ruta de referencia
Backend	backend-vehicular/src/main/java/com/lavarapido/backend_vehicular/
Reservas	reservas/controller/ReservaController.java; reservas/service/ReservaService.java
Pagos	pagos/controller/PagoController.java; pagos/service/PagoService.java
Seguridad	shared/config/SecurityConfig.java; security/JwtService.java
Móvil	lava-rapido-mobile/src/navigation/; src/services/api.ts
Panel administrativo	lava-rapido-admin/src/router/AppRouter.tsx; src/router/RequireAdmin.tsx
Persistencia	changelog-master.yaml; 01_ddl/03_tables/; 01_ddl/09_indexes/; 01_ddl/10_alter/

Anexo B. Lista de comprobación para nuevas ediciones

- Revisiones Git de los componentes documentadas.
- Versiones de dependencias confirmadas en manifests.
- Modelo SQL consistente con changelog y esquema del ambiente.
- Rutas y permisos de API contrastados con código.

- Diagramas y referencias actualizados al alcance elegido.
- Pruebas y evidencias de despliegue fechadas y anonimizadas.